LAPORAN PEMROGRAMAN DASAR DART
BAG. 1 – VARIABEL DAN TIPE DATA
MATA KULIAH PEMROGRAMAN MOBILE
SIB-2D

244107060042_NETY SULISTYORINI

TUGAS PRAKTIKUM

SOAL 01 **Modifikasilah kode pada baris 3 di VS Code atau Editor Code favorit Anda berikut ini agar mendapatkan keluaran (*output*) sesuai yang diminta dengan mengganti sesuai nama Anda!**

Program Asli

```
1 void main() {
2   for (int i = 0; i < 10; i++) {
3     print('hello ${i + 2}');
4   }
5 }
6
```

Output

```
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
```

Program Modifikasi

```
1 void main() {
2   for (int i = 0; i < 10; i++) {
3     print('Nama saya adalah Nety, sekarang berumur ${18 - i}');
4   }
5 }
6
```

Output

```
Nama saya adalah Nety, sekarang berumur 18
Nama saya adalah Nety, sekarang berumur 17
Nama saya adalah Nety, sekarang berumur 16
Nama saya adalah Nety, sekarang berumur 15
Nama saya adalah Nety, sekarang berumur 14
Nama saya adalah Nety, sekarang berumur 13
Nama saya adalah Nety, sekarang berumur 12
Nama saya adalah Nety, sekarang berumur 11
Nama saya adalah Nety, sekarang berumur 10
Nama saya adalah Nety, sekarang berumur 9
```

Kesimpulan

Berdasarkan kode program tersebut dapat di simpulkan bahwa perulangan for yang digunakan pada dart dapat menampilkan output sebanyak 10X, yaitu dari i = 0 sampai i=9. Lalu dengan diubahnya baris print dengan menggunakan aturan 18-i, kode program dapat menampilkan umur yang menurun dari 18 sampai 9. Hal ini menunjukkan bahwa perulangan dan penggunaan tanda ${ } dalam dart dapat digunakan untuk menghasilkan output yang berubah secara otomatis sesuai dengan nilai dari variabel tersebut.

SOAL 02 **Mengapa sangat penting untuk memahami bahasa pemrograman Dart sebelum kita menggunakan framework Flutter ? Jelaskan!**

Hal ini dikarenakan flutter akan bekerja sepenuhnya dengan bahasa dart, dimana untuk mengatur tampilan dan menjalankan fitur nantinya setiap kodenya menggunakan sintaks dari dart. Misalnya adalah konsep dasar variabel, percabangan, perulangan, fungsi, dan lain sebagainya selalu digunakan pada saat membuat aplikasi flutter. Apabila dasar-dasar bahasa pemrograman dart tersebut belum dipahami, secara otomatis akan kesulitan kedepannya baik untuk membaca, memahami, atau membenahi kode program. Selain itu, pesan pemberitahuan error yang akan ditampilkan pada saar kode program flutter sering berasal dari aturan dart. Oleh karena itu, memahami bahasa pemrograman dart sebelum menggunakan framework flutter akan sangat membantu dalam menyelesaikan masalah yang mungkin saja muncul.

SOAL 03 **Rangkumlah materi dari codelab ini menjadi poin-poin penting yang dapat Anda gunakan untuk membantu proses pengembangan aplikasi mobile menggunakan framework Flutter.**

- Bahasa dart pada intinya adalah bahasa yang dibutuhkan oleh kerangka kerja modern seperti flutter, dimana tujuannya yaitu agar bisa memberikan pengalaman terbaik kepada pengembang dan memungkinkan untuk membuat aplikasi seluler yang luar biasa.
- Pemahaman secara mendalam tentang dart adalah dasar untuk bekerja dengan flutter sehingga nantinya dapat mengetahui asal-usul bahasa dart, bagaimana komunitas mengerjakannya, kelebihan dari bahasa dart, dan mengapa ini dipilih sebagai bahasa pemrograman untuk flutter.
- Tujuan dari dart sendiri adalah untuk menggabungkan kelebihan-kelebihan dari sebagian besar bahasa tingkat tingi, adapun fitur-fitur bahasa pemrograman yaitu *productive tooling* (memudahkan coding), *garbage collection* (menghapus memori tidak terpakai), *type annotation* (menentukan tipe data), *statically typed* (mengecek tipe data), dan *portability* (berjalan diberbagai platform).
- Dart adalah bahasa pemrograman modern lintas platform yang awalnya ditujukan untuk menggantikan JavaScript dalam pengembangan web, kemudian berfokus pada mobile development melalui Flutter, dengan keunggulan pada performa tinggi, tooling yang stabil, fleksibilitas tipe data (opsional), serta dukungan OOP sehingga cocok untuk proyek skala besar.
- Dart dapat dieksekusi melalui dua pendekatan utama, yaitu menggunakan Dart Virtual Machine (VM) atau dikompilasi menjadi JavaScript. Fleksibilitas Dart berasal dari kemampuannya menjalankan kode yang sama di berbagai platform tanpa banyak perubahan. Untuk dapat berjalan dengan baik, lingkungan eksekusi Dart memerlukan runtime system, Dart core libraries, serta garbage collector. Dukungan ini memungkinkan Dart digunakan dalam pengembangan lintas platform secara efisien.
- Dart memiliki dua mode kompilasi, yaitu Just-In-Time (JIT) dan Ahead-Of-Time (AOT). Pada mode JIT, kode dikompilasi saat aplikasi dijalankan sehingga sangat cocok digunakan selama proses pengembangan karena mendukung debugging dan hot reload. Sedangkan pada mode AOT, kode dikompilasi sebelumnya menjadi kode mesin native sehingga menghasilkan performa yang lebih tinggi. Mode AOT biasanya digunakan untuk aplikasi yang sudah siap produksi, meskipun tidak mendukung fitur debugging dan hot reload seperti JIT.
- Salah satu keunggulan utama Dart adalah dukungannya terhadap fitur hot reload, yang menjadi fitur andalan dalam framework Flutter. Fitur ini memungkinkan pengembang melihat perubahan kode secara langsung tanpa perlu menjalankan ulang aplikasi, sehingga proses pengembangan menjadi lebih cepat dan efisien. Dengan adanya hot reload, developer dapat memperbaiki kesalahan dan mengembangkan fitur baru dengan lebih responsif, yang pada akhirnya meningkatkan kualitas perangkat lunak yang dihasilkan.
- Dart memiliki sintaks yang mirip C dan JavaScript sehingga mudah dipahami. Dart mendukung built-in types, control flow, dan function seperti bahasa modern lainnya. Selain itu, Dart

berbasis Object-Oriented Programming (OOP) dengan konsep class dan object serta mendukung encapsulation, inheritance, abstraction, composition, dan polymorphism.

- Operator di Dart adalah method dalam class, sehingga semua tipe data dianggap sebagai object. Dart menyediakan operator aritmatika, shortcut (+=, -=, dll.), increment/decrement, equality (==, !=), relational (>, <, >=, <=), dan logical (!, ||, &&). Operator == membandingkan isi data dan sudah type-safe tanpa perlu === seperti di JavaScript.

SOAL 04 **Buatlah penjelasan dan contoh eksekusi kode tentang perbedaan *Null Safety* dan *Late variabel* !**

1. ***Null Safety***

   Null Safety digunakan untuk mengatur apakah sebuah variabel boleh bernilai null atau tidak. Secara default, variabel di Dart tidak boleh bernilai null. Jika ingin mengizinkannya, kita harus menambahkan tanda ? pada tipe datanya.

   - Contoh tanpa nullable

```
//contoh null safety tanpa menggunakan tanda tanya
void main () {
  int angka = 10;
  angka = null; //ERROR
}

```

   Pada kode di atas, angka bertipe int. Karena tidak ada tanda ?, maka variabel tersebut tidak boleh berisi null. Saat kita mencoba mengisi dengan null, program langsung error saat proses kompilasi. Ini menunjukkan bahwa Null Safety membantu mencegah kesalahan sejak awal.

   - Contoh dengan nullable

```
//contoh null safety dengan menggunakan tanda tanya
void main() {
  int? angka;
  angka = null;
  print(angka); //Output: null
}
```

   Di sini, int? berarti variabel boleh bernilai null. Karena itu, saat diisi null tidak terjadi error. Ketika dicetak, hasilnya memang null. Jadi tanda ? berarti "variabel ini boleh kosong".

   - Contoh penggunaan tanda seru (!)

```
//contoh null safety dengan menggunakan tanda seru
void main() {
  int? angka = 5;
  print(angka!);
}
```

   Tanda ! berarti kita yakin nilai tersebut tidak null. Karena memang nilainya 5, program berjalan normal. Namun jika sebenarnya nilainya null dan tetap dipaksa memakai !, maka program akan error saat dijalankan.

2. *Late Variabel*

- Contoh benar

```
//contoh null safety dengan menggunakan kata kunci late
late String nama;

void main() {
  nama = "Dart";
  print(nama);
}
```

Pada contoh ini, variabel nama dibuat tanpa langsung diberi nilai. Namun sebelum dipakai, kita sudah mengisinya dengan "Dart". Karena sudah diisi terlebih dahulu, program berjalan dengan baik.

- Contoh salah

```
//contoh salah penggunaan kata kunci late
late String nama;

void main() {
  print(nama);
}
```

Di sini, nama belum pernah diberi nilai, tetapi langsung dipakai. Akibatnya program error saat dijalankan. Ini terjadi karena late mengharuskan variabel diisi sebelum digunakan.